

AnchorSlice on 200 smart contracts

Claude Opus 5, 200 DIVE contracts, 1,600 label cells, 463 labelled vulnerabilities. Arm 1 reads the complete source with a simple prompt. Arm 2 reads the AnchorSlice output with the calibrated prompt. Same detector, same contracts, one call each, scored against the DIVE labels.

EVIDENCE RETENTION

463 / 463

1.000 in every category, verified before any model call.

DETECTION

F1 0.342 → 0.693

Agreement 0.642 → 0.801, κ 0.097 → 0.549.

TOKENS

-55.2%

10,465 → 4,689 per contract.

COST AND TIME

-63.7% / -59.7%

\$17.75 → \$6.45; 69 min → 28 min.

The two arms

Arm	Detector input	
1 · baseline	complete source	simple: one line per category, "report only what is i
2 · AnchorSlice	slice, 56.5% of characters	calibrated: code example, decision rule and corp

Detection quality

Arm	Agreement	Precision	Recall	F1	Cohen's κ	TP	F
1 · full source, simple prompt	0.642	0.365	0.322	0.342	0.097	149	25
2 · AnchorSlice, calibrated	0.801	0.627	0.775	0.693	0.549	359	21

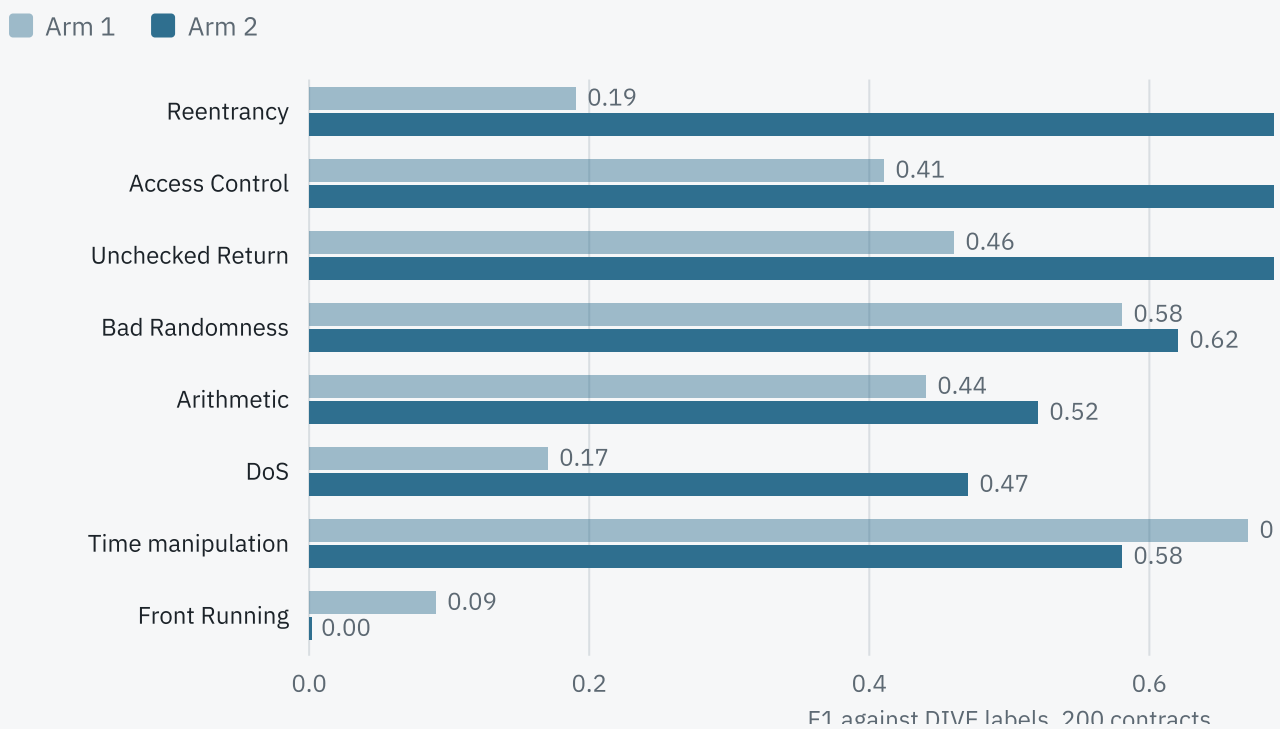
Arm 2 finds **359 of 463** labelled vulnerabilities against 149, cutting misses from 314 to 104, and it does so while also reporting fewer false positives (214 against 259). Precision and recall rise together, so this is not a threshold shift. Per contract, 142 improved, 31 were unchanged and 27 regressed. The paired McNemar test over 1,600 cells is decisive: 170 cells correct only in Arm 1 against 425 correct only in Arm 2, $p \approx 3.7 \times 10^{-26}$.

Token efficiency

Measure	Arm 1	Arm 2	Change
Total tokens	2,092,914	937,754	-55.2%
Tokens per contract	10,465	4,689	-55.2%
Input tokens	1,837,975	825,285	-55.1%
Contract text cached	1,100,024	339,432	-69.1%
Output tokens	254,939	112,469	-55.9%
Cost	\$17.75	\$6.45	-63.7%
Wall clock	4,151 s	1,673 s	-59.7%
Per contract	\$0.089 · 20.8 s	\$0.032 · 8.4 s	-64% · -60%

The slice keeps 56.5% of source characters, and the contract text the detector must read falls by 69%. The longer prompt costs about 1,600 tokens per call but is cached, so it is dwarfed by what the slice removes. Output tokens also fall by 56%: given a compact, tagged input the detector writes shorter justifications. Cost falls faster than tokens because the removed tokens are input, and the slicer itself adds 7.6 ms per contract and needs no compilation.

Per category



Category	Positives	Arm 1 F1	Arm 2 F1	Arm 1 FP/FN	Arm 2 FP/FN
Reentrancy	98	0.19	0.82	0 / 88	39 / 2
Access Control	144	0.41	0.83	17 / 102	46 / 9
Unchecked Return Values	38	0.46	0.74	23 / 20	5 / 13
Bad Randomness	16	0.58	0.62	1 / 9	2 / 8
Arithmetic	84	0.44	0.52	25 / 53	48 / 38
DoS	26	0.17	0.47	50 / 19	10 / 15
Time manipulation	42	0.67	0.58	11 / 15	50 / 4
Front Running	15	0.09	0.00	132 / 8	14 / 15

Six categories improve, two regress. The two largest, Reentrancy (F1 0.19 → 0.82) and Access Control (0.41 → 0.83), drive most of the gain: the simple prompt misses 88 of 98 reentrancy cases and 102 of 144 access-control cases, because "only what is exploitable" makes the detector conservative, while the tags point at the state-mutating entry points where a missing modifier lives.

Front Running is the one real failure. Arm 2 finds none of the 15 positives (F1 0.00), though it also cut the baseline's 132 false positives to 14. The calibrated rule is too strict, and the tightened anchor now tags only 21% of blocks. Arm 1's score on this category is not a real alternative either: it flagged 132 of 185 negatives, so both arms fail, in opposite directions. **Time manipulation** also regressed (0.67 → 0.58): the new rule recovers misses (15 → 4) but triples false positives (11 → 50).

Compared with the analyzers that produced the labels

Detector	Cell coverage	TP	FP	Precision	Recall, where run
AnchorSlice + Opus 5 (Arm 2)	100%	359	214	0.63	0.78
Slither	77%	253	52	0.83*	0.61
Solhint	69%	216	67	0.76*	0.60
VeriSmart	6%	47	0	1.00*	0.55
Mythril	80%	107	2	0.98*	0.27
Semgrep	52%	1	1	0.50	0.00
MAIAN	20%	0	8	0.00	0.00
Arm 1 (full source, simple prompt)	100%	149	259	0.37	0.32

* DIVE's labels are the union of these six tools' output, so each tool is graded against a reference it helped write and its precision is circular by construction. AnchorSlice is scored against a reference it had no part in creating.

Read as coverage rather than a ranking: no analyzer answers every category on every contract — coverage runs from 6% to 80%, and two of the six contribute almost nothing here. Reading a sliced contract, the detector reproduces the six-tool consensus on 80% of all cells and recovers 78% of its positives at complete coverage, for \$0.032 and 8 seconds per contract.

What the paper can claim

RQ1 • Evidence retention. 463 of 463 labelled vulnerabilities retained across 200 contracts, 1.000 in every category, at 56.5% of source characters and 7.6 ms per contract, with no compilation. Reaching this needed four slicer fixes that the earlier 70-contract sample never exposed: legacy functions that are public by default, multi-line signatures, anonymous fallbacks, and an over-eager veto rule.

RQ2 • Token efficiency. 55% fewer tokens, 64% lower cost and 60% less wall-clock time per contract, with the contract text read down 69%.

RQ3 • Detection effectiveness. F1 0.342 → 0.693, agreement 0.642 → 0.801, κ 0.097 → 0.549, misses 314 → 104 and false positives 259 → 214 at the same time; McNemar $p \approx 3.7 \times 10^{-26}$ over 1,600 paired cells. Token cost falls and detection improves together, which is the paper's thesis.

Limitations to state

Arm 2 changes two things at once, the input and the prompt, so this experiment measures the combination; the earlier 70-contract runs showed slicing alone is token-neutral for detection ($p = 1.00$), and the prompt carries most of the accuracy gain. The calibrated rules describe how DIVE's labels behave rather than Solidity semantics — each category is effectively defined by whichever analyzer ran — so they are calibration to this benchmark and must be declared as such. The rules were derived from 15 contracts inside this set, which is in-sample for those; on the 130 contracts added afterwards the slicer, not the prompt, was re-verified. Front Running (0 of 15) and Time manipulation (50 false positives) are open failures, and Arithmetic remains the weakest large category at F1 0.52. Sampling over-represents the two rare categories (Bad Randomness 8.0% against 2.8% corpus-wide, Front Running 7.5% against 2.7%). Each arm ran once; three repeats of one configuration earlier varied by 4.2 points of agreement.

Data: `final200/arm1_simple_full.json`, `final200/arm2_v5_slice2.json`; slicer `slice2.py`; prompt `calibrated5` in `run_eval.py`; contracts in `selected_ids_50.txt`, `selected_ids_20new.txt`, `selected_ids_130.txt`.